

Mobility Diary: A Context-Aware System for Personal Mobility Diaries with Human Activity Recognition

Giacomo Bianco

Department of Computer Science and Engineering

University of Bologna

Bologna, Italy

giacomo.bianco4@studio.unibo.it

Abstract—This paper presents Mobility Diary, a context-aware platform that reconstructs a personal mobility diary from smartphone GPS traces and inertial data. A human activity recognition model identifies the user’s mode of travel, while recurring locations are automatically grouped into habitual places. A spatial perturbation technique protects the user’s location by producing both a detailed private view and a less precise, shareable view. The resulting tradeoff between privacy and data quality is evaluated experimentally.

I. INTRODUCTION

Understanding how people move through urban environments is an increasingly important problem, with applications ranging from transportation planning to the design of personalized services. Location tracking alone, however, cannot capture the meaning of a journey. A sequence of GPS coordinates does not reveal whether a user traveled on foot, by bicycle, or by car, nor whether a visited location is personally significant, such as the user’s home or workplace. A context-aware mobility system must therefore go beyond the acquisition of raw location data. By enriching a trace with recognized activities and significant places, it can turn a sequence of GPS points into a genuinely informative mobility diary.

This paper presents Mobility Diary, a context-aware system comprising a mobile application that collects mobility data, a backend that handles processing and persistence, and a web dashboard for viewing the diary. The system combines Human Activity Recognition (HAR) with mechanisms that protect the privacy of location data. Specifically, the system provides:

- an asynchronous ingestion pipeline for raw data collected by the mobile device. It uses direct uploads to object storage (MinIO), followed by explicit confirmation, so that raw data does not pass through the application server. Asynchronous processing on the server then materializes the mobility trace;
- reconstruction of the user’s mobility as a sequence of temporal segments, each associated with a place or movement activity (walking, cycling, traveling by car, or stopping), based on traces collected by the device;
- a human activity recognition module based on a pipeline with two stages: a CNN1D and a bidirec-

tional GRU, trained on the public SHL (Sussex-Huawei Locomotion) dataset [1]. It recognizes five mobility classes (IDLE, WALKING, RUNNING, BIKING, and MOVING_VEHICLE) with 84.88% accuracy on the official validation set;

- three diary views with different privacy levels (*precise*, *approximate*, and *aggregated*), produced by applying spatial generalization on a grid (spatial cloaking) to GPS traces, together with a quantitative evaluation of the tradeoff between location perturbation and quality of service;
- a web dashboard through which authorized personnel can view mobility diaries and directly compare a true trace with the trace perturbed according to the selected privacy level.

The remainder of this paper is organized as follows. Section II describes the system architecture, Section III details its implementation, and Section IV presents the experimental results.

II. SYSTEM ARCHITECTURE

A. Overview

The system has two clients, a mobile application and a web dashboard. Both reach the backend through a single gateway, as shown in Fig. 1. The backend handles both synchronous diary operations and asynchronous processing of raw data uploaded by the device, as detailed in the following subsections. Data originates on the user’s device, is uploaded to the backend, and undergoes asynchronous processing to extract contextual information, including recognized activities and significant places. It is then persisted and made available to the web dashboard, either with or without privacy protection.

B. Mobile Application

The mobile application, developed in Flutter, collects data on the user’s device. A finite state machine driven by sensor data independently determines whether the user is moving or stationary. This allows the application to adapt its sampling rate to the current state and avoid continuous acquisition at a

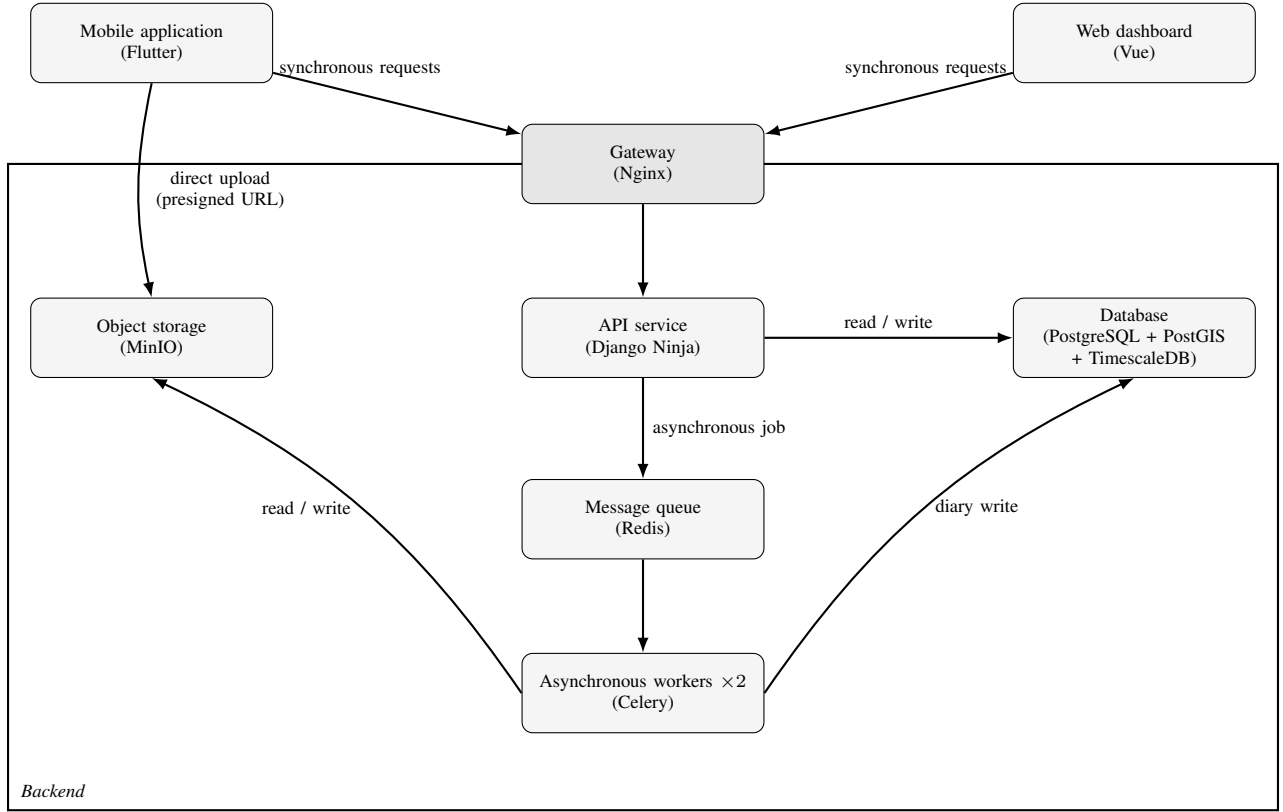


Fig. 1. Overall system architecture

high sampling rate, which would consume excessive battery power. Section III describes this state machine in detail.

C. Gateway and Application Service

Synchronous requests from the mobile application and web dashboard pass through a single Nginx gateway, which routes traffic to the application service. The service is implemented with Django and Django Ninja and replicated across two instances to handle concurrent requests.

This layer exposes the system’s main synchronous operations: mobile and dashboard authentication; retrieval of daily diaries and mobility traces; confirmation, rejection, and labeling of significant places; personal statistics; privacy preferences; diary exports with privacy protection; and user management through the dashboard. It also schedules operations that require asynchronous processing in the message queue.

D. Ingestion and Processing Pipeline

Raw data collected by the device is not uploaded through the application service. Instead, the device uploads it directly to object storage and then explicitly confirms completion to the backend. This design prevents large transfers from occupying application service connections and exhausting its capacity to handle concurrent requests. The service only authorizes the upload and confirms its completion; the data transfer occurs directly between the device and object storage.

Once an upload is confirmed, the asynchronous processing layer (Celery), decoupled from the application service through a Redis message queue, begins processing the data. Processing consists of four sequential stages, each dependent on the output of the previous one.

1) *Activity Classification*: This stage labels the sensor data windows stored in object storage with the recognized mode of travel.

2) *Diary Insertion*: This stage materializes the mobility trace, its segments, and the movement labels from the uploaded data and the labels produced by the previous stage.

3) *Batch Insertion of Sensor Data*: This stage persists the raw sensor measurements in TimescaleDB.

4) *Habitual Place Analysis*: This stage identifies significant places visited by the user from the location data inserted into the diary.

The asynchronous nature of these four stages makes it possible to handle computationally expensive workloads, particularly stages 1 and 3, without affecting the latency of synchronous requests to the application service. Section III discusses the implementation of each stage.

E. Persistence

The system uses two persistence mechanisms selected according to the nature of the data. Structured diary data, including traces, segments, significant places, and activity labels, is stored in a relational database extended with PostGIS [2] and

TimescaleDB [3]. These extensions support geographic queries and computationally intensive analytical queries based on aggregation. Raw sensor data does not need to be queried sample by sample; it is retrieved as a whole during asynchronous processing. It is therefore kept in object storage, separately from the database.

F. Data Model

The entities required by the project specification are represented in the persistent model under the names shown in Table I. Separating traces, segments, and places preserves the raw evidence and allows the system to reconstruct a current diary view whenever place labels change.

The system also stores inertial samples in `RawSensorReading`, state machine transitions in `StateTransition`, and asynchronous processing status in `HarJob` and `TripUpload`. These are operational entities and do not alter the domain vocabulary required by the specification.

G. Dashboard

The Vue web dashboard is intended for administrators rather than end users. It allows an administrator to select a user from a list and inspect that user's mobility diary for support and verification. It consumes the synchronous APIs exposed by the application service to display the selected user's mobility traces and significant places on a map, the daily diary in tabular and timeline views, and aggregate statistics for the activities recognized during the pipeline's data mining stage. It also allows administrators to compare the true trace with its corresponding protected view and filter the displayed data by day, recognized activity, trip, or time interval.

H. Containerization

The entire server platform can be started with Docker Compose. An Nginx gateway acts as the sole HTTP entry point. It routes `/api`, `/admin`, and static files to Django, uploaded objects to MinIO, and all remaining requests to the Vue application. PostgreSQL with PostGIS, Redis, and the Celery workers complete the stack.

III. IMPLEMENTATION

A. Mobile Application

The mobile application follows Pine, an architecture for Flutter projects that separates the UI, state management logic (Cubit/BLoC), repositories, and external services into distinct layers, with dependencies flowing in one direction [9]. The following subsections describe the application modules built on this structure.

1) *Finite State Machine for Movement Detection*: The application is designed to run for several hours throughout the day, making energy consumption a significant design constraint. While running, it maintains three sensor streams: accelerometer, gyroscope, and GPS. To limit energy consumption without sacrificing data quality, the application runs a local finite state machine that determines whether the user

is moving (`movement`) or stationary (`stationary`) (Section II). Each state has a different sampling profile, allowing the application to reduce the acquisition rate while the user is stationary.

To decide whether to change state, the machine observes two values: the instantaneous speed reported by GPS and σ , the standard deviation of the accelerometer signal vector magnitude (SVM). For each sample, the SVM is $\sqrt{x^2 + y^2 + z^2}$ across the three acceleration components. Rather than recalculating σ for every accelerometer sample, the system accumulates samples in a window and computes it once when the window is complete. A window contains 20 samples in `stationary` (2 s at 10 Hz) and 500 samples in `movement` (5 s at 100 Hz, matching the window used for HAR classification).

At the start of a trip, the machine is initialized in the `stationary` state. Its sampling profile sets the accelerometer to 10 Hz and disables the gyroscope. In `movement`, the gyroscope is enabled at 100 Hz and contributes to sensor windows for HAR classification. In both states, GPS does not sample at a fixed frequency; it emits a new fix when the user moves at least 3 m from the last known position. A smaller displacement threshold would produce an excessive number of fixes relative to GPS accuracy. All of those fixes would be persisted, increasing storage requirements without a proportional improvement in trace quality.

Table II summarizes the sampling rate of each sensor in both states and the amount of data required to trigger a new state machine evaluation. Each such event recalculates the evidence (moving, stationary, or uncertain) from the newly arrived data.

The accelerometer and GPS therefore operate as two independent, asynchronous processes. Whenever either produces new data, the machine reevaluates its current state by combining that value with the most recent value from the other signal. Freshness checks of 10 s and 20 s, respectively, treat a value older than its threshold as unavailable. The evaluation yields moving, stationary, or uncertain evidence by comparing σ and GPS speed with fixed thresholds.

The thresholds vary by situation. A GPS speed of at least 2.5 m/s is sufficient on its own to indicate movement, regardless of σ , because such a speed clearly indicates movement. Below this threshold, when the current state is `movement`, the decision depends only on σ : values of at least 1.2 indicate movement, values below 0.8 indicate a stationary user, and intermediate values are uncertain.

σ takes priority for two related reasons. When the machine is in `movement` and must determine whether the user has stopped, the stop often occurs indoors, where GPS is less accurate. Moreover, when the user is truly stationary, the 3 m distance filter causes GPS to stop emitting new data. GPS therefore becomes both less accurate and less informative precisely when confirmation of a stop is needed, whereas the accelerometer remains reliable.

When the current state is `stationary`, the state machine instead requires agreement between σ and GPS speed. Movement evidence requires $\sigma \geq 1.2$ and speed ≥ 0.8 m/s;

TABLE I
MAPPING BETWEEN CONCEPTUAL TRACE ENTITIES AND THE IMPLEMENTED MODEL

Conceptual entity	Implemented model	Meaning and main attributes
MobilityTrace	Trip, GpsPoint	A Trip delimits a mobility session and stores its status, time interval, distance, and LineString path. Each GpsPoint stores a timestamp, PostGIS coordinates, speed, and measurement accuracy.
MobilitySegment	MobilitySegment	A persistent diary entry of type MOVE or STOP, with start and end times, recognized activity, distance, and an optional path.
SignificantPlace	HabitualPlace, CandidateVisit	A CandidateVisit represents a single stay. Spatially recurring visits are aggregated into a HabitualPlace, with a center, radius, counts, review status, category, and a name defined by the user.
ActivityLabel	ActivityLabel	A controlled vocabulary shared by segments and the classifier: IDLE, WALKING, RUNNING, BIKING, and MOVING_VEHICLE.

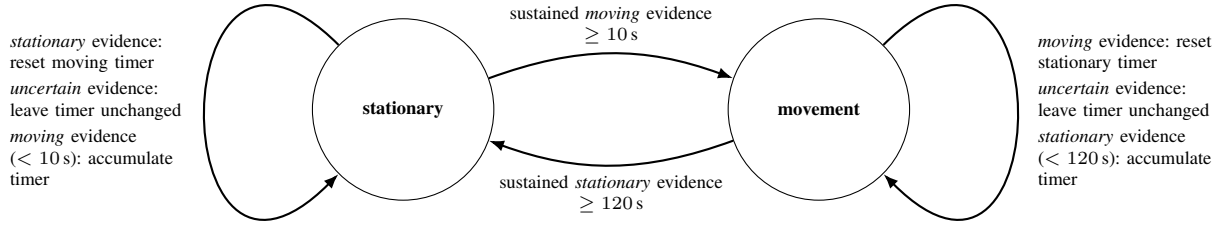


Fig. 2. Finite state machine for movement detection: states, transitions, and the effect of each evidence type on the hysteresis timers

TABLE II
SENSOR SAMPLING RATES AND EVENT GENERATION FOR THE STATE MACHINE

Sensor	Stationary	Movement	Triggers an evaluation after
Accelerometer	10 Hz	100 Hz	20 samples (2 s) in stationary; 500 in movement (5 s)
Gyroscope	off	100 Hz	does not drive the FSM; HAR windows only
GPS	every 3 m	every 3 m	immediately, at every fix

stationary evidence requires $\sigma < 0.8$ and speed < 0.5 m/s. All other combinations remain uncertain. This stricter requirement reduces false positives caused by a single noisy signal while the user is stationary. In particular, the system is designed not to report small indoor displacements, such as a few steps at home or in a gym, as movement.

State transitions are also subject to an asymmetric minimum confirmation time: evidence must persist before the state change is confirmed. This provides another filter against brief movements, which rarely last long enough to exceed the threshold. A transition from *stationary* to *movement* requires 10 s of sustained movement evidence, whereas the reverse transition requires 120 s of sustained stationary evidence. This asymmetry prevents a short stop during a journey, for example at a traffic light, from prematurely ending a movement

segment.

At each transition, the machine updates the sensor sampling profile. When the state becomes *movement*, the GPS filter remains unchanged, but the gyroscope is enabled and both inertial sensors move to 100 Hz. This rate matches the SHL data used to train the Human Activity Recognition model (Section III), because the sensor windows collected in this state are later classified by the model. Fig. 2 summarizes the states, transitions, and evidence cases handled by the machine.

2) *Relationship between the State Machine and HAR:* The state machine and the Human Activity Recognition system do not carry equal weight when constructing the diary: the state machine is the primary source of truth. A time interval that it classifies as *stationary* is accepted as a stop in the final diary, and HAR does not alter such segments. HAR only classifies the mode of travel within intervals that the state machine has already classified as *movement*.

This hierarchy directly determines what is persisted locally on the device. Accelerometer and gyroscope windows intended for HAR classification are written to the local SQLite database only while the state machine reports *movement*. In *stationary*, only accelerometer data is available because the gyroscope is disabled. This data is used solely by the state machine and does not produce windows for persistence. Nevertheless, HAR may identify subintervals during which the user is actually stationary within an interval labeled *movement*,

such as a brief stop at a traffic light. Section III discusses this case later.

GPS data follows a different policy: it is recorded throughout the trip, including during stops. GPS points are always persisted because they are needed to reconstruct the complete trip, including its map representation. For a trip comprising three consecutive segments, Fig. 3 summarizes which data is persisted in SQLite during each phase before it is uploaded and incorporated by the backend through the ingestion pipeline described in Section II-D.

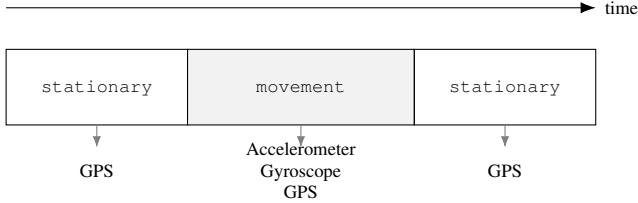


Fig. 3. Data persisted locally in SQLite during a trip with three segments. Only GPS data is stored during *stationary* phases; the accelerometer and gyroscope data used for HAR classification are also stored during the *movement* phase.

3) *Structure and Packaging of Sensor Data for HAR*: As described in Section II, the accelerometer and gyroscope both sample at 100 Hz in the *movement* state. Each 5 s interval is grouped into a 500×6 window: 500 time samples for the six components *Acc_x*, *Acc_y*, *Acc_z*, *Gyr_x*, *Gyr_y*, and *Gyr_z*. Each window is stored locally as one row in the SQLite *SensorWindows* table. The matrix is serialized as JSON in the *matrixJson* column, together with metadata such as start and end timestamps, sample count, and sampling rate.

During synchronization, windows are neither uploaded individually nor combined into a single file. Instead, they are divided among multiple files. The application reads rows from the database in pages of 128 windows and accumulates them in a buffer until the uncompressed JSON size exceeds 12 MB. It then writes that buffer to a file and opens a new one. Each uncompressed window occupies approximately 29 KB. JSON is a text format, and every value is first rounded to six decimal places. For example, a single row containing six channels is represented as follows:

```
[−0.023457, −9.806649, 0.112346,
0.001235, −0.004321, 0.000765]
```

With values in this form, one row occupies approximately 58 characters: 51 for the six numbers and 7 for commas and brackets. This corresponds to roughly 58 UTF-8 bytes, because each digit and symbol occupies one byte. Across 500 rows, one window therefore occupies approximately 29,000 bytes. A 12 MB file holds about 430 windows, or approximately 36 minutes of recording. This deliberately conservative threshold limits peak serialization memory on the phone regardless of the total trip duration.

Each file is then compressed with GZIP, a lossless format based on the DEFLATE algorithm. It identifies repeated data and encodes it more compactly, which is effective for highly

regular JSON structures. Compression reduces the file size by more than half, from 12 MB of uncompressed data to about 5 MB. Both compression and SHA-256 checksum calculation run in an isolate, Flutter’s mechanism for executing code on a separate thread. This prevents an expensive operation from blocking UI rendering.

The checksum is used during upload confirmation (Section III). The application sends the locally computed SHA-256 string to the backend, which compares it with the checksum of the file actually received in object storage and verifies that they match.

4) *Offline Synchronization Queue*: Locally collected data is synchronized with the backend through the *SyncJobs* table in the local SQLite database. Its records form a synchronization queue: each collection session produces one job, and jobs are processed individually in chronological order, oldest first, to avoid saturating the connection with multiple concurrent sessions.

Each job is uploaded in two phases. During the *core* phase, GPS points and state changes between *movement* and *stationary* are sent directly to the API service without passing through object storage. During the *raw* phase, the files described in the previous subsection are uploaded using the direct upload and confirmation pattern from Section II-D. The backend first authorizes each part; the device then uploads it and confirms it with the corresponding checksum. Each job tracks the two phases separately, so they can be confirmed and retried independently. Up to three raw parts are uploaded in parallel to reduce total synchronization time.

The queue explicitly handles cases in which the backend is still processing received data asynchronously. It places the job on hold and checks its status again at regular intervals. After an error, it retries the job with increasing delays of 10 s, 30 s, 2 min, and 10 min, up to a maximum of five attempts. If the backend has already created a trace for a discarded job, that trace is also canceled on the server to avoid orphaned partial data. After successful completion, temporary local files are deleted and the session is removed from the local database, limiting storage usage on the device.

5) *Single Active Trip Constraint*: The system is designed around a single timeline for each user. The diary is a sequence of consecutive trips, so two trips cannot be in progress simultaneously. To enforce this constraint, the backend creates a trip record, including its start time and associated device, as soon as the user presses Start, even though no collected data is sent at that point. GPS traces and sensor data are not uploaded incrementally during recording; they are uploaded together only after the user presses Stop, through the synchronization queue described above.

Creating the record at the outset makes enforcement possible. If the user attempts to start another trip from a different device while the same account already has an unfinished trip, the backend rejects the request and prevents the two recordings from overlapping on the same timeline.

During a trip, the application also sends a heartbeat to the backend every 5 minutes, updating a freshness value associated

with the active trip. This mechanism handles the edge case in which a trip appears to be active but has not received updates for an extended period. If no heartbeat has arrived for more than 24 hours, the backend automatically closes the stale trip when the user next attempts to start one, whether from the same device or another device. Recording can then resume instead of remaining indefinitely blocked by an inactive trip.

6) *Progressive Display of Results*: Given the two upload phases (*core* and *raw*) and the four asynchronous processing stages described in Section II-D, the mobile application does not wait for the entire pipeline before displaying a newly recorded trip. As soon as the *core* phase is confirmed, it displays the complete trip map built from the GPS points uploaded in that phase. This is the first useful information available from the backend.

To display the diary with segments colored by mode of travel, the application polls the backend once per second for the trip's processing status. The backend reports the diary as complete only after the first two stages of the asynchronous pipeline have finished: activity classification and diary insertion. Stages 3 and 4, batch insertion of sensor data and habitual place mining, are not required for display and continue independently. The user therefore waits only for the first two stages, rather than for all asynchronous processing, before seeing the result of the trip. This substantially reduces perceived loading time. Fig. 4 summarizes the pipeline from the user pressing Stop through completion of data mining, together with the two points at which the interface visibly changes.

7) *Background Operation*: The application is designed for prolonged use throughout the day: the user presses Start and leaves it running, often in the background, for several hours. It must therefore handle both suspension by the operating system and the simpler case in which the user does not reopen it for several minutes. Data collection is not equally reliable for all sensors under these conditions. Background location permissions allow the application to continue receiving GPS fixes, but the accelerometer and gyroscope libraries cannot guarantee continuous samples under every background condition imposed by the operating system.

When the user returns after an absence, the application compares the timestamps of the latest locally recorded state transition, GPS point, and sensor window. The most recent of these is treated as the last known activity time. If at least 30 minutes have elapsed, the session is considered to have ended automatically. It is closed retroactively, using the last known activity rather than the reopening time as its end time, and sent to the backend as though the trip had ended then.

If less than 30 minutes have elapsed, the session resumes normally, with one correction. If the last known state was *movement* but sensor data stopped arriving during the interval, that state is no longer considered reliable because no recent evidence confirms that the user is still moving. The system forces the state to *stationary* from the point at which data stopped, and the state machine resumes from this corrected state rather than the last observed state.

This mechanism on the mobile device is not sufficient on its own. The state machine may still classify a segment as *movement* even though no sensor data is available for that interval. The backend therefore provides an additional safeguard. If a movement segment contains no classified HAR windows during diary construction, the system does not leave it unlabeled. Instead, it estimates the activity from the median GPS speed in the interval, using fixed thresholds: walking below 8 km/h, running up to 13 km/h, cycling up to 25 km/h, and traveling by vehicle above that value.

B. Context-Aware Route Assistant

The route assistant uses the current GPS position as point A and lets the user search for and select point B through Mapbox geocoding. The request [4] uses one of three profiles: walking, cycling, or driving. The GeoJSON response is displayed on the map together with estimated distance and duration. The profile can be selected manually or derived from HAR: WALKING and RUNNING map to walking, BIKING maps to cycling, and MOVING_VEHICLE maps to driving.

In live mode, the client retrieves the latest 500×6 inertial window every 15 seconds and sends it to the classification endpoint. If the recognized mode changes, the client state is updated and the route is requested again with the corresponding profile. Because the new request uses the most recent position, the route updates in response to changes in both travel mode and location. Generation counters associated with geocoding and routing requests prevent stale asynchronous responses from overwriting a newer selection.

C. Activity Classification (HAR)

1) *Dataset and Preparation*: The HAR module is trained on the public Sussex-Huawei Locomotion (SHL) dataset [1], collected using Huawei smartphones worn simultaneously at four body positions: pocket, bag, torso, and hand. At each position, accelerometer, gyroscope, and magnetometer data were recorded at 100 Hz and divided in advance into windows of 500 samples (5 s).

The dataset's eight original classes (IDLE, WALKING, RUNNING, CAR, BIKING, BUS, TRAIN, and SUBWAY) were reduced to five by merging the four motorized transport modes into MOVING_VEHICLE. Confusion among these modes was common because the inertial signatures produced by cars and public transit are very similar. Merging them turns such errors into correct predictions without discarding information useful to the system, which does not need to distinguish among motorized modes.

The training protocol uses the official training set, reserves its last 15% in chronological order for early stopping, and uses the official validation set exclusively for final evaluation.

2) *Model Architecture*: The base classifier is a 1D convolutional neural network (CNN1D) that operates along the time axis [5], reading every sensor channel at each step. It contains two blocks that combine convolution and pooling (Conv1D(64) and Conv1D(128), kernel size 10, with

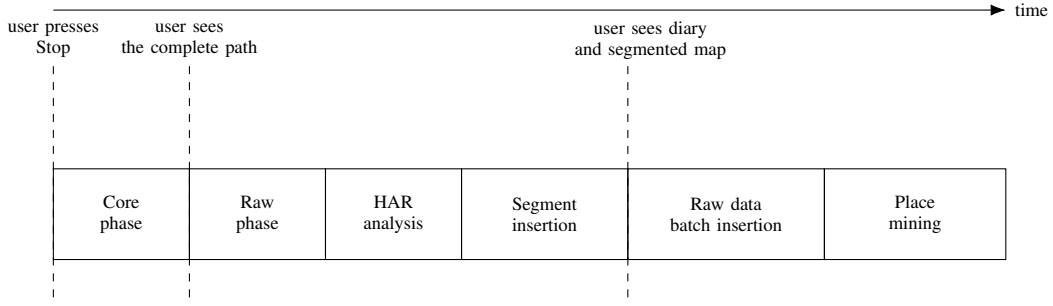


Fig. 4. Complete upload and processing pipeline

MaxPooling1D), followed by a Dense(128) layer and a softmax output layer for five classes, yielding approximately 2 million parameters in total.

The input is the raw 500×6 accelerometer and gyroscope window. The convolutional filters learn discriminative local patterns, and the dense layer compresses them into an embedding with 128 values.

A gated recurrent unit (GRU) [6] is a recurrent neural network designed to process sequences. Unlike the CNN, which classifies each 5 s window independently, the GRU maintains an internal state that is updated from one window to the next, conditioning each prediction on neighboring windows. Its reset and update gates control how much information from the previous state is retained or discarded. This allows it to capture dependencies across relatively long sequences with fewer parameters than an equivalent LSTM.

The system uses a bidirectional GRU, which reads the embedding sequence in both directions so that each prediction can use both preceding and subsequent context. This explains its use in addition to the CNN. An isolated window may look the same for a vehicle stopped in traffic and a stationary person. The surrounding windows, however, may show driving in the first case and walking or inactivity in the second. Section IV-B discusses this ambiguity in detail. The temporal context supplied by the GRU makes the distinction possible.

D. Diary Insertion and Sensor Data Persistence

1) *Geospatial Representation and Distance Queries*: All geospatial data, including GPS points, segment paths, and the centroids of visits and habitual places, is stored in the database using the PostGIS geography type (SRID 4326), rather than planar geometry. This allows PostGIS to account for the Earth's curvature instead of treating latitude and longitude as planar coordinates. In particular, the length of a path stored as a LineString can be obtained with a single function (ST_Length on geography), which directly returns a geodesic distance in meters. The application therefore does not need to implement formulas for distances between coordinates. The system uses this function to compute the distance of each new movement segment, compare a path with its protected counterpart (Section III), and obtain physical distances from stored geometries more generally.

2) *Diary Segmentation*: During the first segmentation stage, the system uses state machine transitions (Section III) to identify coarse STOP/MOVE intervals. STOP intervals become stop segments without further processing because the state machine is already the source of truth for stops. MOVE intervals are refined in a second step using the HAR labels assigned to sensor windows within each interval.

This refinement applies two corrections at different time scales to distinguish classifier noise from genuine pauses in movement. The first corrects short, isolated label changes. If a group of windows with a different label lies between two groups with the same label and lasts less than 60 s, it is reassigned to the surrounding label on the assumption that it is a classification error rather than a genuine activity change.

The second correction distinguishes genuine stops from spurious ones within a movement interval. It addresses an inherent limitation of the state machine: the machine requires at least 120 s of sustained stationary evidence before reporting a stop (Section III). During those 120 s, the device continues collecting and classifying sensor windows. HAR is likely to label them correctly as IDLE because the user has already stopped, even though the coarse segment generated by the state machine remains labeled movement until its hysteresis period ends.

A sufficiently long group of IDLE windows is therefore recognized as a genuine stop and becomes a *virtual stop*. A shorter IDLE group is absorbed into the surrounding movement label as classifier noise. Finally, temporally adjacent groups with the same label are merged into one segment (Fig. 5).

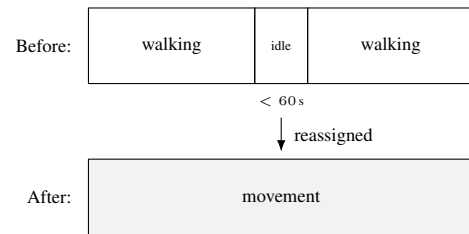


Fig. 5. Correction of a short, isolated label change (duration < 60 s): the central group is reassigned to the surrounding label.

The distinction between a virtual stop and a state machine

stop remains internal to segmentation. In the final diary shown to the user, both are handled identically and merged into a single stop (Fig. 6).

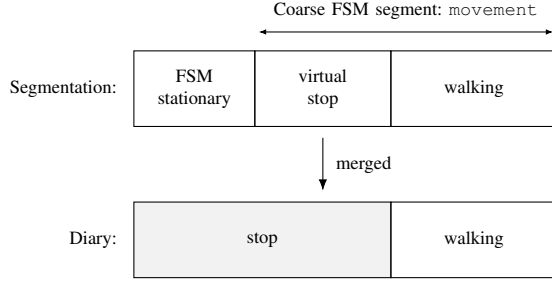


Fig. 6. A virtual stop detected within a coarse movement segment is merged with the adjacent stationary stop determined by the FSM when the final diary is constructed.

3) *Raw Sensor Data Persistence*: After diary construction is complete (stages 1 and 2 of the pipeline, Section II-D), the third stage inserts the raw sensor data uploaded by the device into the time series database. Each accelerometer and gyroscope sample becomes a separate row.

Insertion uses `bulk_create` in batches of 5,000 rows. Storing each sample as a separate row supports the analytical queries described below.

4) *Analytical Use of Raw Sensor Data*: Storing one raw sample per database row makes the data directly queryable with SQL. The system uses this capability for two analyses available in the administration panel.

The first query detects abnormal gaps in an individual trip's recording. It compares every accelerometer and gyroscope sample with the preceding sample in the same movement segment and reports intervals longer than 10 ms, given the 100 Hz sampling rate.

For each recognized activity class, the second query calculates the mean and standard deviation of all six accelerometer and gyroscope components across all of the user's trips.

E. Habitual Place Mining

Habitual places are identified in two successive stages: stay detection over raw GPS points, followed by spatial clustering of those stays into recurring places using DBSCAN. Stay detection is necessary because DBSCAN groups points only by spatial proximity and does not consider time. On its own, it cannot distinguish points that are dense because the user remained in one place for a long time from points that are dense because the user returned there on different days.

The first stage processes all of a user's GPS points in chronological order, discarding points with accuracy worse than 60 m. It accumulates consecutive points in a cluster while each new point remains within 75 m of the current centroid and no more than 90 minutes have passed since the previous point. A point that is too distant or follows an excessive time gap closes the current cluster and starts a new one. A cluster becomes a candidate *visit* only if it contains at least three

points and spans at least 5 minutes, thereby excluding very short stops.

The detected visits are then spatially clustered with DBSCAN [7]. For each resulting cluster, the system computes its centroid, its radius (the maximum distance between the centroid and a visit in the cluster), and the number of distinct days on which a visit occurred. A DBSCAN cluster becomes a candidate habitual place only if its visits span at least two distinct days. It is automatically confirmed when its visits span at least four days; otherwise, it awaits manual review by the user through the dashboard. Fig. 7 summarizes the complete pipeline from raw GPS points to the habitual place lifecycle.

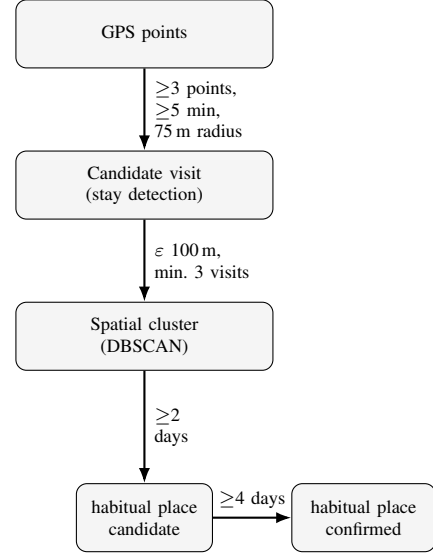


Fig. 7. Habitual place detection pipeline, from raw GPS points through stay detection and DBSCAN clustering to the candidate and confirmed place lifecycle.

Confirmed places enrich the diary when it is retrieved. Each stop segment, whether a state machine stop or a virtual stop, is associated with the confirmed place nearest to its GPS centroid, if one exists.

This dependency affects how the private diary is served. Strictly speaking, there is no definitive diary computed in advance to return, because its content also depends on the user's habitual places. Those places can change independently of the trip, for example when the user confirms, rejects, or relabels a place in the dashboard. The private diary is therefore cached in Redis, but its cache key includes both the trip identifier and a version counter for the user's places. Every place revision increments the counter, making the old cache entry unreachable through the new key without requiring explicit invalidation.

F. Privacy Module

Of the three diary views (*precise*, *approximate*, and *aggregated*; Section II), only the latter two provide protection. The *precise* view applies no transformation and exposes GPS coordinates exactly as collected. It is therefore not a true

privacy level, but the user’s personal, unprotected view of their data.

The *approximate* and *aggregated* views use spatial cloaking [8]. Each GPS coordinate is projected into Web Mercator (EPSG:3857) and assigned to a square grid. The cells measure 150 m in the *approximate* view and 400 m in the *aggregated* view. The coordinate is moved to the center of its cell and then transformed back into geographic coordinates, as shown in Fig. 8.

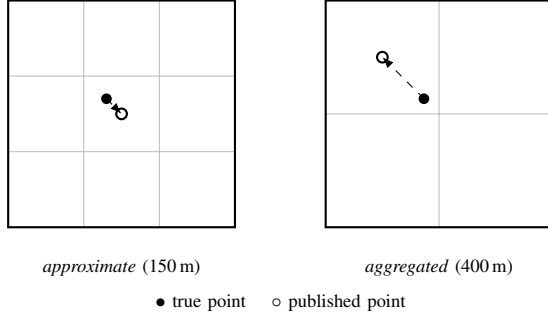


Fig. 8. Spatial generalization on a grid. The true point is published at the center of its cell. A larger cell (*aggregated*) can move the published point farther from the true point.

Fig. 9 shows the same mechanism on a real trip. It compares the original trace (solid line) with the corresponding *approximate* view (dashed line), obtained by replacing every point with the center of its cell and connecting consecutive centers with straight segments.

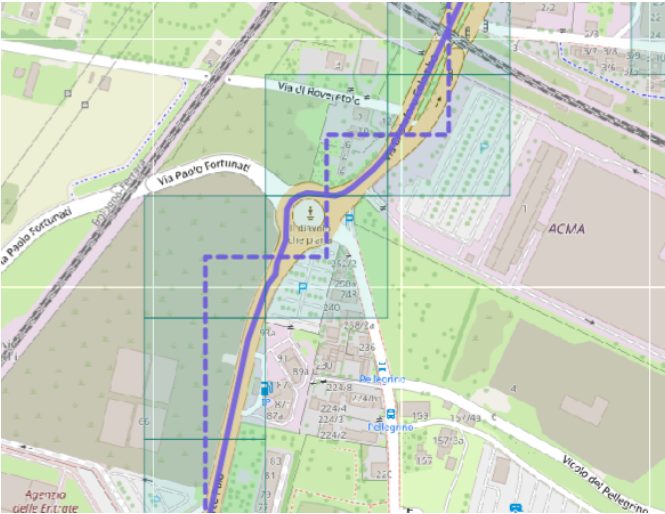


Fig. 9. Example using real data from a car trip: the solid line is the recorded path, and the dashed line is its *approximate* view (150 m grid).

1) *Diary Construction by Privacy Level*: The three privacy levels affect the geographic representation and the detail used to describe places. Segment start and end times remain unchanged at every level. The *precise* view does not transform the data: each segment shows its exact start and end times, the recorded GPS path, and therefore the actual distance traveled.

A stop also shows the true name of its associated habitual place, if any.

In the *approximate* view, the displayed path is the result of the grid approximation described above rather than the recorded path. The segment distance is therefore recalculated from this approximate path. The name associated with a stop remains visible, as in the precise view. Because coordinate approximation already protects the location, displaying its name does not significantly increase the risk.

The *aggregated* view applies the same path transformation with a larger grid (400 m rather than 150 m) and also conceals place identities. Instead of the habitual place name, it shows only a generic category, or a neutral label if the stop is not associated with a confirmed place.

IV. RESULTS

A. HAR Evaluation

The evaluations use the official SHL validation set [1], which was never used to update model weights. Table III compares the two evaluation configurations.

TABLE III
COMPARISON OF CNN+GRU VARIANTS ON THE OFFICIAL VALIDATION SET

Variant	Channels	Sequence	Accuracy
Original configuration	6	32	79.47%
Deployed configuration	6	128	84.88%

Both variants share the same CNN+GRU checkpoint; only the sequence length used for evaluation changes, not the weights.

Because GRU weights do not depend on input sequence length, the layer applies the same transformation at every step regardless of the number of steps in the sequence. The same checkpoint was therefore evaluated at several sequence lengths to determine whether the improvement from 32 to 128 windows continued, stabilized, or reversed at greater lengths. Table IV and Fig. 10 report the measured trend on the official validation set of 115,156 windows.

TABLE IV
ACCURACY OF THE DEPLOYED MODEL AT DIFFERENT SEQUENCE LENGTHS

Sequence length	Accuracy
32	79.46%
64	82.50%
128	84.88%
256	86.08%
512	86.38%

The improvement is substantial between 32 and 128 windows (+5.4 percentage points) and then progressively diminishes (128→256: +1.2 points; 256→512: +0.3 points) without reversing. Accuracy continues to increase up to 512 windows (approximately 43 minutes of context), although with diminishing marginal returns. Thus, within this range, neither the anticipated plateau nor the slight decline beyond 256 windows occurs. The production sequence length of 128 is therefore a

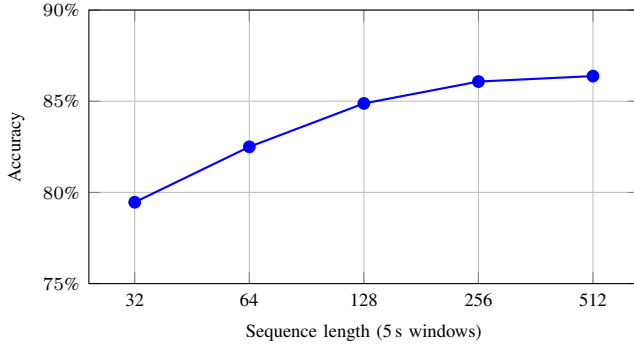


Fig. 10. Accuracy on the official validation set by GRU input sequence length, with identical weights and no retraining

compromise driven by latency rather than the point at which accuracy saturates. This choice reflects the bidirectional GRU’s nonlinear scaling on a CPU.

This accuracy improvement alone does not justify increasing L in production. Latency was measured directly with the production code and model (HARClassifier). All measurements used the same warm process at each sequence length and covered realistic request sizes.

TABLE V
HARCLASSIFIER LATENCY BY SEQUENCE LENGTH (MS)

L	100	500	1000	2000	Warmup
32	81.7	131.0	567.0	921.7	7.2 s
64	87.4	211.3	659.4	1186.0	15.0 s
128	130.9	265.9	507.0	1212.7	30.4 s
256	345.2	404.1	947.0	1479.1	73.2 s
512	605.1	518.3	982.5	1299.5	166.2 s

The startup cost (*warmup*, which compiles the graph once per worker process) increases from 30.4 s at 128 windows to 166.2 s at 512, a factor of $5.5\times$. For the smallest request size (100 windows), warm latency increases from 130.9 ms to 605.1 ms, a factor of $4.6\times$. Given these costs, the accuracy gain of 1.5 percentage points at 512 windows compared with 128 does not justify adopting $L = 512$ in production. The choice of $L = 128$ is supported by direct latency measurements rather than by evidence that accuracy had already plateaued.

The deployed model was evaluated on 115,156 windows. Before inference, the raw values of the six channels were restored from the normalized tensor.

Sequences were constructed separately for each of the four body positions so that GRU context never crossed independent recordings.

Accuracy is 84.88%. Macro precision, recall, and F1 are 87.67%, 77.12%, and 80.08%, respectively; the corresponding weighted averages are 85.60%, 84.88%, and 84.30%.

The dominant error is confusion between `IDLE` and `MOVING_VEHICLE`. A vehicle stopped at a traffic light or in traffic can produce a signal similar to that of a stationary person.

The `BIKING` class has a recall of 0.414 and is confused mainly with `WALKING` and `MOVING_VEHICLE`. `RUNNING`

has a precision of 0.998 but a recall of 0.754, partly because of its limited support.

B. Discussion of Ambiguous Cases

The confusion matrix (Table VII) supports a quantitative analysis of cases that accuracy alone would conceal.

1) *Vehicle Stopped in Traffic or at a Traffic Light*: This is the most common ambiguous case: 3,009 true `IDLE` windows are classified as `MOVING_VEHICLE`, while 6,346 true `MOVING_VEHICLE` windows are classified as `IDLE`. In a single 5 s window, the inertial signal from a stopped vehicle is indistinguishable from that of a stationary person. The distinguishing information, a positive average speed over the surrounding minutes, exists only at the sequence level, not in an isolated window.

2) *Temporary Stops during Movement*: A brief stop, such as waiting at a crossing while walking, may produce isolated `IDLE` windows within an otherwise `WALKING` sequence. At the level of a single window, this is a local error consistent with the preceding analysis. Diary segmentation (Section III) does not carry this error into the final diary as a separate stop. Any differently labeled group lasting less than 60 s is reassigned to its surrounding label (Fig. 5). An isolated window therefore cannot produce a spurious stop in the final diary.

C. Privacy and Utility Tradeoff

The Privacy Perturbation and Quality of Service metrics defined in Section III were computed segment by segment over the trips recorded in the database. Movement segments were grouped by recognized activity; stop segments were evaluated separately, with perturbation computed from the stop’s GPS centroid rather than from a path length. Table VIII reports the results.

As expected, Privacy Perturbation increases with cell size. Moving from *approximate* (150 m) to *aggregated* (400 m) more than doubles the mean distance between true and published points in every mode, including stops. Quality of Service, however, does not always worsen monotonically with aggregation. For vehicle segments, the relative distance error is slightly lower with larger cells (0.61) than with smaller cells (0.64). The cycling sample is too small ($n=3$) to support robust conclusions and is included only for completeness.

D. Ingestion Pipeline Latency

A load test using a synthetic client that reproduces the behavior of the mobile application (Section III) measured the time spent in each stage of the trip upload and processing pipeline. The test simulated 25 users concurrently uploading trips of approximately one hour each. Because each user can have only one active trip at a time (Section III), each user uploaded 10 trips sequentially, for a total of 250 trips. Raw upload time was measured on the client; the other four stages were measured on the backend. The test ran in a local environment equivalent to production, using the same Docker containers, but not on the production hardware. The absolute values should therefore be treated as indicative, whereas the relative proportions among stages remain informative.

TABLE VI
METRICS BY CLASS FOR THE DEPLOYED CNN+GRU MODEL USING SIX CHANNELS

Class	Support	Precision	Recall	F1
IDLE	23,856	0.737	0.855	0.792
WALKING	20,920	0.848	0.946	0.894
RUNNING	2,220	0.998	0.754	0.859
BIKING	9,624	0.906	0.414	0.569
MOVING_VEHICLE	58,536	0.894	0.887	0.890

TABLE VII
CONFUSION MATRIX FOR THE DEPLOYED CNN+GRU MODEL USING SIX CHANNELS; TRUE CLASSES BY ROW AND PREDICTED CLASSES BY COLUMN

True \ predicted	IDLE	WALK	RUN	BIKE	VEHICLE
IDLE	20,395	346	0	106	3,009
WALKING	626	19,786	3	92	413
RUNNING	0	332	1,674	213	1
BIKING	301	2,581	0	3,989	2,753
MOVING_VEHICLE	6,346	285	0	2	51,903

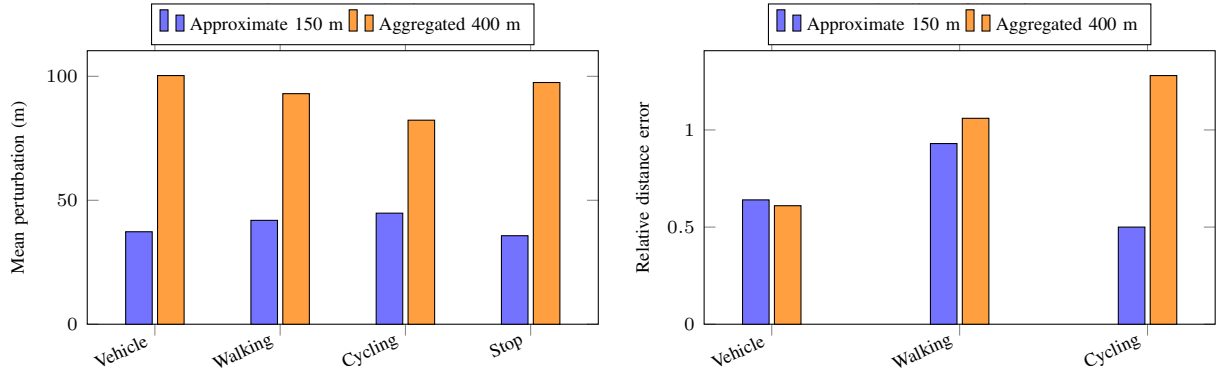


Fig. 11. Measured tradeoff between location protection and utility. The larger grid always increases perturbation, whereas distance distortion depends on the geometry and typical length of segments in each mode.

TABLE VIII
MEAN PRIVACY PERTURBATION AND QUALITY OF SERVICE (RELATIVE DISTANCE ERROR) BY MODE FOR THE TWO PROTECTED LEVELS

Mode	n	Perturbation (m)		QoS (relative error)	
		approx.	aggr.	approx.	aggr.
Vehicle	38	37.3	100.3	0.64	0.61
Walking	56	41.9	93.0	0.93	1.06
Cycling	3	44.8	82.3	0.50	1.28
Stop	51	35.7	97.5	N/A	N/A

TABLE IX
MEAN LATENCY OF INGESTION PIPELINE STAGES (250 TRIPS, 25 CONCURRENT USERS)

Stage	Mean	Std. dev.	Min	Max
Core (GPS+transitions)	2.02 s	2.61 s	0.25 s	15.83 s
Raw upload (client)	3.04 s	2.56 s	0.43 s	28.59 s
HAR + diary	4.62 s	1.50 s	1.99 s	11.06 s
Raw batch insertion	29.68 s	8.235 s	20.39 s	53.97 s
Place mining	0.97 s	0.64 s	0.27 s	2.74 s

Batch insertion of raw sensor data is by far the most expensive stage, averaging 29.68 s and approaching one minute in the worst case. The cost arises from the number of rows written to `RawSensorReading`: one per sample, or up to 360,000 rows for a trip lasting one hour (720 windows of 500 samples). Insertion already uses `bulk_create` in groups of

5,000 rows, resulting in 72 batches for a trip lasting one hour. The cost therefore comes not from one query per row but from the absolute amount of data written at this granularity, compounded by database contention among 25 concurrent users.

Batch insertion of raw data is not visible to the user. The trip diary is available as soon as the HAR stage completes, while persistence of raw data continues in the background. From the user's perspective, latency includes only the core upload, raw upload, and HAR and diary stages. It therefore remains low overall, on the order of a few seconds.

The speed of the HAR stage is partly due to the model being loaded into memory only once when each Celery worker process starts (`warm_har_model_on_worker_start`, connected to the `worker_process_init` signal), rather than for every inference request. This makes runtime inference fast, but each worker consumes a substantial amount of RAM because it holds the model in memory continuously.

These results suggest two approaches to scaling the system beyond simply increasing the worker count. First, replacing the model copy in each worker with a single shared instance exposed as a separate service would reduce aggregate memory usage at the cost of slightly higher inference latency for each request. This is a favorable tradeoff when the goal is to run

more workers concurrently on the same machine. Second, the database was the most heavily loaded component during the test, even more so than the Celery workers. Horizontal scaling would therefore require database replication and synchronization among replicas, in addition to replicating application workers.

Declaration on the Use of Artificial Intelligence Tools

Claude (Anthropic) was used in two distinct areas during the development of this project.

1) *Comparison and Research*: The tool supported research and comparisons among ideas and design choices. In particular, it accelerated the collection and comparison of multiple sources and approaches, including their respective advantages and disadvantages.

2) *Code Development*: The tool was used to:

- help develop the final user interfaces for the mobile application and web dashboard;
- refactor and simplify existing code, particularly in the frontend and mobile components, to align it with the architecture selected for the project;
- write small portions of backend code that handle edge cases;
- write frontend data transfer objects (DTOs) that map JSON responses from the backend REST APIs to the data models used by the mobile application and web dashboard.

CONCLUSION

This paper presented Mobility Diary, a context-aware system for constructing personal mobility diaries. The platform comprises a mobile application, a backend that handles asynchronous processing, and an administrative dashboard. The system combines a finite state machine for movement detection, a human activity recognition module based on a CNN1D and a GRU, and spatial generalization mechanisms that protect the user's location through three privacy levels.

After optimization for production inference, the HAR module achieves 84.88% accuracy on the official validation set. The privacy analysis over recorded trips shows that Privacy Perturbation increases predictably with the size of the grid cell used for generalization, whereas the effect on data utility depends on the travel mode. Quality of Service remains nearly constant under the larger grid for typically long, straight vehicle trips, while aggregation causes a more pronounced loss of utility for shorter walking paths.

The ingestion pipeline load test shows that the latency perceived by users remains on the order of a few seconds. This interval covers data upload and HAR classification, after which the diary becomes available. The considerably slower persistence of raw sensor data runs in the background and has no perceptible impact. Under concurrent load, the database is the most heavily stressed component. Future scaling would therefore require database replication, as well as a shared HAR model rather than a copy held in each worker's memory.

The system nevertheless has limitations. Its movement detector uses an ad hoc heuristic built around a finite state machine and based on accelerometer and GPS data. It is likely less accurate than the native activity recognition APIs provided by mobile operating systems, which were not used. The HAR module also does not distinguish among motorized transport modes such as cars, buses, and trains; all are assigned the single `MOVING_VEHICLE` label.

Two directions for future development are particularly relevant. The first is a periodic process that deletes a trip's raw sensor data from object storage after processing is complete. Currently, this data is never removed automatically and remains in storage indefinitely unless the user explicitly deletes the trip. The second is a mechanism that allows users to correct erroneous HAR labels. For example, a user could report that a segment labeled as walking was actually traveled by car. A related pipeline could collect these corrections and periodically update the model weights.

REFERENCES

- [1] "Sussex-Huawei Locomotion & Transportation Dataset." [Online]. Available: <http://www.shl-dataset.org/>. Accessed: Sep. 14, 2026.
- [2] PostGIS Project, "PostGIS Manual: Geography Data Type and ST_Length." [Online]. Available: https://postgis.net/docs/using_postgis_dbmanagement.html and https://postgis.net/docs/ST_Length.html. Accessed: Sep. 13, 2026.
- [3] "TimescaleDB: An Introduction to Time-Series Databases," *Data Engineering and Algorithms (Medium)*. [Online]. Available: <https://medium.com/dataengineering-and-algorithms/timescaledb-an-introduction-to-time-series-databases-3438d275e88e>. Accessed: Sep. 14, 2026.
- [4] Mapbox, "Geocoding API and Directions API," *Mapbox API Documentation*. [Online]. Available: <https://docs.mapbox.com/api/search/geocoding/> and <https://docs.mapbox.com/api/navigation/directions/>. Accessed: Sep. 13, 2026.
- [5] A. Jain, "Understanding the 1D Convolutional Layer in Deep Learning," *Medium*, Jan. 7, 2025. [Online]. Available: <https://medium.com/@abhishekjainindore24/understanding-the-1d-convolutional-layer-in-deep-learning-7a4cb994c981>. Accessed: Sep. 14, 2026.
- [6] "Gated Recurrent Unit Networks," *GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/gated-recurrent-unit-networks/>. Accessed: Sep. 14, 2026.
- [7] S. Yıldırım, "DBSCAN Clustering: Explained," *Towards Data Science*. [Online]. Available: <https://towardsdatascience.com/dbscan-clustering-explained-97556a2ad556/>. Accessed: Sep. 14, 2026.
- [8] "Spatial Cloaking," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Spatial_cloaking. Accessed: Sep. 14, 2026.
- [9] A. Avvenuti, "Pine: A Lightweight Architecture Helper for your Flutter Projects," *Medium*. [Online]. Available: <https://angeloavv.medium.com/pine-a-lightweight-architecture-helper-for-your-flutter-projects-1ce69ac63f74>. Accessed: Sep. 14, 2026.